



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SE/SS ZG566

Secure Software Engineering

T V Rao



- *The slides presented here are obtained from the authors of the books, product documentations, and from various other contributors. I hereby acknowledge all the contributors for their material and inputs.*
- *I have added and modified slides to suit the requirements of the course.*



Phases in software development

Software Engineering - Definitions

- (1969 – Fritz Bauer) Software engineering is the establishment and use of *sound engineering principles* in order to obtain *economically* software that is *reliable* and works *efficiently* on *real machines*
- (IEEE) The application of a *systematic, disciplined, quantifiable* approach to the *development, operation, and maintenance* of software; that is, the application of engineering to software

Knowledge Areas in v4(2024) of SWEBOK

Requirements	Configuration Management
Architecture	Quality
Design	Security
Construction	Processes
Testing	Models & Methods
Maintenance	Engineering Management
	Project Management
	Economics

Architecture and Security are new additions in v4

Some Authors refer two columns as Primary & Supporting Activities
Pressman calls them Framework and Umbrella Activities

Software Engineering Process KA

A Process defines who is doing what, when, and how to reach a certain goal

-Ivar Jacobson, Grady Booch, and James Rumbaugh

A software process is a set of interrelated activities and tasks that transform input work products into output work products. At minimum, the description of a software process includes required inputs, transforming work activities, and outputs generated.

- SWEBOK

A software process infrastructure can provide process definitions, policies for interpreting and applying the processes, and descriptions of the procedures to be used to implement the processes.

A software development life cycle (SDLC) includes the software processes used to specify and transform software requirements into a deliverable software product.

How Process Models Differ?

While all Process Models take same primary and supporting activities, they differ with regard to

- Overall flow of activities, actions, and tasks and the interdependencies among them
- Degree to which actions and tasks are defined within each framework activity
- Degree to which work products are identified and required
- Manner in which quality assurance activities are applied
- Manner in which project tracking and control activities are applied
- Overall degree of detail and rigor with which the process is described
- Degree to which customer and other stakeholders are involved in the project
- Level of autonomy given to the software team
- Degree to which team organization and roles are prescribed

Prescriptive and agile processes

Prescriptive processes are processes where all of the process activities are planned in advance and progress is measured against this plan.

In agile processes, planning is incremental and it is easier to change the process to reflect changing customer requirements.

In practice, most practical processes may include elements of both plan-driven and agile approaches.

There are NO right or wrong software processes.

Software Requirements

The Software Requirements knowledge area (KA) is concerned with the elicitation, analysis, specification, and validation of software requirements as well as the management of requirements during the whole life cycle.

Software projects are critically vulnerable when the requirements related activities are poorly performed

Functional & Nonfunctional Requirements

Functional requirements describe the functions that the software is to execute; for example, formatting some text or modulating a signal. They are sometimes known as capabilities or features. A functional requirement can also be described as one for which a finite set of test steps can be written to validate its behavior.

Nonfunctional requirements are the ones that act to constrain the solution. Nonfunctional requirements are aka constraints or quality requirements. They can be classified according to whether they are performance requirements, maintainability requirements, safety requirements, reliability requirements, security requirements, interoperability requirements

Emergent properties of software

Some requirements represent *emergent properties* of software—that is, requirements that cannot be addressed by a single component but that depend on how all the software components interoperate.

—The throughput requirement for a call center would, for example, depend on how the telephone system, information system, and the operators all interacted under actual operating conditions.

Emergent properties are crucially dependent on the system architecture.

Experts consider ***Security*** as an ***emergent property*** of the software.

Software Design

Software design consists of two activities that fit between software requirements analysis and software construction:

- Software architectural design (sometimes called high-level design): develops top-level structure and organization of the software and identifies the various components.
- Software detailed design: specifies each component in sufficient detail to facilitate its construction.

Software Design

- Software design principles include abstraction; coupling and cohesion; decomposition and modularization; encapsulation/information hiding; separation of interface and implementation; sufficiency, completeness, and primitiveness; and separation of concerns.
- Design for security is concerned with
 - How to prevent unauthorized disclosure, creation, change, deletion, or denial of access to information and other resources.
 - It is also concerned with how to tolerate security-related attacks or violations by limiting damage, continuing service, speeding repair and recovery, and failing and recovering securely.

Software Construction

Software construction refers to the detailed creation of working software through a combination of coding, verification, unit testing, integration testing, and debugging.

Throughout construction, software engineers both unit test and integration test their work. Thus, the Software Construction KA is closely linked to the Software Testing KA.

Code is the ultimate deliverable of a software project, and thus the Software Quality KA is closely linked to the Software Construction KA.

Software Testing

Software testing consists of the *dynamic* verification that a program provides *expected* behaviors on a *finite* set of test cases, suitably *selected* from the usually infinite execution domain

- *Dynamic*: The input value alone is not always sufficient to specify a test, since a system might react to the same input with different behaviors, depending on the system state.
- *Finite*: Even in simple programs, so many test cases are theoretically possible that exhaustive testing is infeasible.
- *Selected*: How to identify the most suitable test set under given conditions is a complex problem; in practice, risk analysis techniques and software engineering expertise are applied.
- *Expected*: It must be possible, although not always easy, to decide whether the observed outcomes of program testing are acceptable or not.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Software development – Supporting Activities

Software quality

Software quality may refer:

- to desirable characteristics of software products,
- to the extent to which a particular software product possess those characteristics, and
- to processes, tools, and techniques used to achieve those characteristics

Experts defined variously

“conformance to requirements” - Phil Crosby

“achieving excellent levels of fitness for use” - Watt Humphrey

“market-driven quality” where the “customer is the final arbiter” - IBM

Software quality

A healthy software engineering culture includes the understanding that tradeoffs among cost, schedule, and quality are a basic tenant of the engineering of any product. The tradeoff is best decided by understanding four cost of quality categories: prevention, appraisal, internal failure, and external failure.

Prevention costs include investments in software process improvement efforts, quality infrastructure, quality tools, training, audits, and management reviews

Appraisal costs arise from project activities that find defects.

Costs of internal failures are those that are incurred to fix defects found during appraisal activities and discovered prior to delivery of the software product to the customer

External failure costs include activities to respond to software problems discovered after delivery to the customer.

Configuration management

Configuration management (CM) is the discipline of identifying the configuration of a system at distinct points in time for the purpose of systematically controlling changes to the configuration and maintaining the integrity and traceability of the configuration throughout the system life cycle.

A system can be defined as the combination of interacting elements organized to achieve one or more stated purposes

Configuration of a software system is collection of specific versions of hardware, firmware, or software items combined according to specific build procedures for a purpose.

Software Maintenance

A software product must change or evolve over time. Once a software is in operation, defects are uncovered, operating environments change, and new user requirements surface.

Software maintenance is an integral part of a software life cycle. However, software development used to receive more importance than software maintenance in most organizations. It is changing

- Due to large capital spending needed for software development, organizations are trying to maximize lifespan of existing software
- Due to large scale availability of open source components, organizations increased focus on maintenance

Maintainer's activities

Five key characteristics comprise the maintainer's activities:

- maintaining control over the software's day-to-day functions;
- maintaining control over software modification;
- perfecting existing functions;
- identifying security threats and fixing security vulnerabilities; and
- preventing software performance from degrading to unacceptable levels.

Software Engineering Management

Software engineering management can be defined as the application of management activities — planning, coordinating, measuring, monitoring, controlling, and reporting — to achieve quality etc.

There are aspects specific to software projects and software life cycle that complicate effective management, including

- Clients often don't know what is needed or what is feasible
- As a result of changing requirements, software is often built using an iterative process
- The degree of novelty and complexity is often high
- There is often a rapid rate of change in the underlying technology

Software Security

Security has become a significant issue in software development because of potential misuse and increasing malicious activity targeting computer systems.

- Build security by following a set of established and/or recommended rules and practices.
- Secure maintenance complements secure software dev by ensuring that no security problems are introduced during software maintenance
- Identified vulnerabilities, which are errors that attackers can exploit, can be handled during the software life cycle. Vulnerabilities are not only introduced at the development, but also by
 - Libraries, COTS, or OS.

Software Engineering Economics

Software engineering economics is about relating the attributes of software and software processes to economic measures

- involves balancing risk and profitability, while maximizing benefits and wealth of the organization.
- identify organizational goals, time horizons, risk factors, and financial constraints
- identify and implement the appropriate portfolio and investment decisions to manage cash flow, and funding;
- measure financial performance, such as cash flow and ROI

Software Engineering Process

software process is a set of interrelated activities and tasks that transform input work products into output work products

a software process includes required inputs, transforming work activities, and outputs generated

a software process may also include its entry and exit criteria and decomposition of the work activities into tasks, which are the smallest units of work

Value individuals & interactions over processes & tools – Agile manifesto

Agile models are designed to facilitate evolution of the software requirements during the project

Software Engineering Professional Practice

Software Engineering Professional Practice includes knowledge, skills, and attitudes that software engineers must possess to practice software engineering in a professional, responsible, and ethical manner

The concept of professional practice becomes applicable within the professions that have a generally accepted body of knowledge

A code of ethics and professional conduct for software engineering was approved by the ACM Council and the IEEE CS Board of Governors in 1999

–Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the eight principles concerning the public, client and employer, product, judgment, management, profession, colleagues, and self, respectively



Software Engineering: A Practitioner's Approach, 7/e Roger Pressman

Software Engineering, Pearson Education, 9th Ed., 2010. Ian Sommerville

SWEBOK by IEEE/ACM

www.owasp.com

sei.cmu.edu

www.microsoft.com

www.devopedia.org/devops

www.ibm.com



Thank You!